这是一份为你量身定制的本地开发 README.md 指南。你可以直接将其复制到你毕设项目的根目录下作为项目文档。它完全基于你的任务书要求，结合了 DSperse 的模型切片架构 1 和 EZKL 的零知识证明生成流程 2，并详细列出了实现步骤、所需链接和评估指标的收集方法。

DistriZKP: 面向分布式推理的零知识证明框架设计与低开销优化

📖 项目简介

本项目为本科/硕士毕业设计项目。随着模型规模扩大，集中式零知识机器学习（ZKML）面临严重的计算和内存瓶颈 3, 4。本项目基于 **“模型切片（Model Slicing）”** 与 **“定向验证（Targeted Verification）”** 的核心思想 1，实现了一个原生支持分布式推理的零知识证明框架。
通过将完整的神经网络模型切分为多个连续的子图（Slices），并将计算分摊至分布式网络中的多个工作节点（Workers） 5，本项目显著降低了单节点的内存负载与证明生成时间。同时，通过引入相邻节点间的**哈希状态承诺链（Hash Chain）**，系统能够在不泄露模型参数和原始数据的前提下，实现 100% 的恶意节点篡改检测，确保分布式推理的完整性 3。

🛠 技术栈与依赖链接

- **深度学习框架**: PyTorch (用于模型训练与模型切分)
- **模型中间态**: ONNX (用于导出独立的子模型切片) 6
- **零知识证明引擎**: EZKL (基于 Halo2，提供极简的 Python API 将 ONNX 转为 ZK 电路) 7
- **分布式通信协议**: Python FastAPI + Requests 或 gRPC
- **资源监控**: Python psutil 库 (用于收集内存与 CPU 开销) 8
- **参考架构基座**: DSperse / zkML Blueprints 9

⚙️ 环境配置 (Local Setup)

建议在 Linux (Ubuntu) 或 Windows WSL2 环境下使用 Conda 进行环境隔离。

```
# 1. 创建虚拟环境
conda create -n distrizkp python=3.9 -y
conda activate distrizkp

# 2. 安装深度学习库 (根据你的 CUDA 版本调整)
pip install torch torchvision torchaudio

# 3. 安装核心 ZK 引擎 EZKL
pip install ezkl

# 4. 安装通信与监控依赖
pip install fastapi uvicorn requests psutil
```

📂 推荐的项目目录结构

```
DistriZKP/
│
├── models/                    # 存放 PyTorch 模型定义与切片脚本
│   ├── train_baseline.py      # 训练完整 LeNet/ResNet 模型
│   └── slice_model.py         # 将完整模型切分为 slice1.onnx,
slice2.onnx ...
│
├── orchestrator/              # 主节点 (Master) 调度中心
│   ├── master_node.py         # 负责分发任务、收集 Proof、哈希链比对与批量
验证
│   └── verifier.py            # 调用 ezkl.verify 验证收集到的零知识证明
│
├── worker/                    # 工作节点 (Prover)
│   ├── worker_node.py         # 接收输入，执行本地 ONNX 推理，调用
ezkl.prove
│   └── fault_injector.py      # 故障注入脚本 (用于模拟恶意节点篡改数据)
│
├── benchmark/                 # 指标评估脚本 (用于生成论文图表数据)
│   ├── measure_memory.py      # 使用 psutil 监控单节点内存占用
│   └── measure_latency.py     # 测试端到端推理延迟与系统吞吐量
│
└── README.md                  # 本文档
```

🚀 核心开发步骤指南

步骤 1：模型切片与 ONNX 导出 (Model Slicing)

目标：打破集中式 ZKP 的内存墙，将计算图沿网络层边界切分 10。

1. 在 models/slice_model.py 中，加载你训练好的 PyTorch 模型（如 LeNet-5）。
2. 将其拆分为两个独立的 nn.Module。例如：Slice1 包含前两个卷积层，Slice2 包含全连接层 11。
3. 使用 torch.onnx.export 将它们分别导出为 slice1.onnx 和 slice2.onnx。

步骤 2：开发 Worker 节点 (本地推理与 ZKP 生成)

目标：在接收到上游数据后，执行推理并生成局部证明。在 worker/worker_node.py 中，编写 FastAPI 接口接收输入 Tensor。节点需顺序执行以下 EZKL 步骤（参考 EZKL 官方文档）：

1. ezkl.gen_settings(): 根据当前切片的 ONNX 生成电路配置。
2. ezkl.compile_circuit(): 编译电路。
3. **前向推理**: 使用 ezkl.forward() 或 onnxruntime 得出这一层的激活值输出。
4. ezkl.prove(): 为这一步计算生成零知识证明 proof.json 12。
5. **哈希计算**: 计算输出 Tensor 的 SHA-256 哈希值，随同 Proof 一起返回给调度器。

步骤 3：开发 Master 节点 (调度与状态承诺绑定)

目标：协调分布式节点，并通过“哈希链”防范恶意节点 3。

1. Master 节点将初始图像发送给 Worker 1。
2. 接收 Worker 1 的中间结果（Hidden States）、哈希值与 Proof 1 13。
3. 将中间结果路由给 Worker 2。
4. **【安全核心】**: Master 必须强制检查 Hash(Worker 1 的真实输出) == Hash(Worker 2 证明中的公开输入 (Public Input))。

步骤 4：恶意节点检测与故障注入 (Fault Injection)

目标：完成任务书中的“恶意节点检测准确率”指标 14。

1. 在 worker/fault_injector.py 中，模拟一个被攻陷的节点。让它在计算完中间结果后，故意给某个张量数值加上 0.5 噪声，然后强行用假数据生成 Proof。
2. 观察 Master 节点。由于哈希断链或 EZKL 验证不通过，系统应准确拦截该次请求并抛出警报，记录拦截率（预期为 100%）。

📊 实验评估与指标收集指南 (Evaluation)

依据任务书要求，你需要收集以下 6 大关键指标，请运行 benchmark/ 目录下的脚本进行对比（集中式全模型 vs. 分布式切片模型）：

- **单节点 CPU / 内存占用率 (Peak Memory)**:
- *方法*: 使用 Python psutil.Process().memory_info().rss 监控调用 ezkl.prove() 期间的内存峰值 8。
- *预期*: 分布式切片将使单节点的内存需求下降一个数量级（例如从数 GB 降至几十 MB）15。
- **证明生成时间 (Proving Time) & 验证时间 (Verification Time)**:
- *方法*: 使用 Python time.time() 记录各个 Worker 并行生成 Proof 的耗时，以及 Master 批量验证的时间 16。
- *预期*: 切片后总体生成时间显著缩短 17。
- **推理延迟 (Inference Latency) & 系统吞吐量 (Throughput)**:
- *方法*: 记录从输入图像到最终输出结果的端到端耗时（包含网络通信与序列化开销）。吞吐量通过并发请求测试计算 (Requests per second)。
- **恶意节点检测准确率**:
- *方法*: 运行故障注入测试 100 次，统计成功识别恶意篡改的次数除以总攻击次数。

📚 重要参考文档与链接

- **EZKL Python API 教程**: EZKL Jupyter Notebooks (重点参考如何将 ONNX 转化为 Circuit)
- **模型切分理论参考 (DSperse)**: DSperse: A Framework for Targeted Verification in Zero-Knowledge Machine Learning (arXiv) 18
- **Halo2 协议背景**: The Halo2 Book 19

导师/开发者提示：

- 在开发初期，务必先在单机的一个 Python 脚本里，跑通一个极其简单的 2 层 MLP 模型的 EZKL 证明与验证（对应 Baseline），不要一上来就加 RPC 分布式通信。
- **避坑指南：**EZKL 涉及浮点数到有限域整数的量化（Quantization）。在 ezkl.gen_settings() 中调整 scale 参数时，请务必留意模型精度的变化 20。